

PrintWizard: Next-Level Trace-Based Debugging

Master's Thesis Presentation · SYSTEMF

29.01.2026

Kelvin Kappeler

Shardul Chiplunkar · Clément Pit-Claudel

Coding is fun.

Developers can build amazing things, solve complex problems, and bring ideas to life.

Coding is fun.

Developers can build amazing things, solve complex problems, and bring ideas to life.



Developers are human.

And humans make mistakes.

Developers are human.

And humans make mistakes.



Even the most experienced developers are susceptible to make mistakes while coding.

Bugs are inevitable.

Bugs are inevitable.

- Human factors

Bugs are inevitable.

- Human factors
- Requirements misunderstandings

Bugs are inevitable.

- Human factors
- Requirements misunderstandings
- Integration problems

Bugs are inevitable.

- Human factors
- Requirements misunderstandings
- Integration problems
- Data issues

Bugs are inevitable.

- Human factors
- Requirements misunderstandings
- Integration problems
- Data issues
- ...

Bugs are inevitable.

- Human factors
- Requirements misunderstandings
- Integration problems
- Data issues
- ...

? How to fix bugs within your program?

Example #1.

```
import java.util.List;

public class Main {
    static double average(List<Integer> xs) {
        int sum = 0;
        for (int i = 0; i < xs.size(); i++) sum += xs.get(i);

        return sum / xs.size();
    }

    public static void main(String[] args) {
        var xs = List.of(1, 2, 2);
        System.out.println("avg = " + average(xs));
    }
}
```

Example #1.

```
import java.util.List;

public class Main {
    static double average(List<Integer> xs) {
        int sum = 0;
        for (int i = 0; i < xs.size(); i++) sum += xs.get(i);

        return sum / xs.size();
    }

    public static void main(String[] args) {
        var xs = List.of(1, 2, 2);
        System.out.println("avg = " + average(xs));
    }
}
```

```
avg = 1.0
```

Example #1.

```
import java.util.List;

public class Main {
    static double average(List<Integer> xs) {
        int sum = 0;
        for (int i = 0; i < xs.size(); i++) sum += xs.get(i);

        return sum / xs.size();
    }

    public static void main(String[] args) {
        var xs = List.of(1, 2, 2);
        System.out.println("avg = " + average(xs));
    }
}
```

```
avg = 1.0
```

Problem: `avg` prints 1.0 but we expect 1.666...

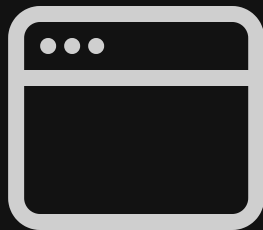
How to fix this bug?

Two main strategies:

How to fix this bug?

Two main strategies:

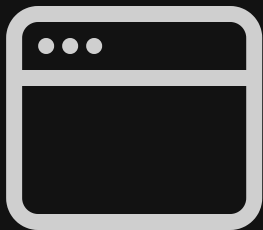
Logging



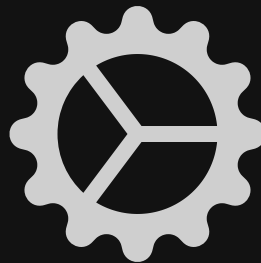
How to fix this bug?

Two main strategies:

Logging



Traditional debugger



Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    for (int i = 0; i < xs.size(); i++)  
        sum += xs.get(i);  
    return sum / xs.size();  
}
```

Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    System.out.println("xs = " + xs);  
  
    for (int i = 0; i < xs.size(); i++)  
        sum += xs.get(i);  
  
    return sum / xs.size();  
}
```

Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    System.out.println("xs = " + xs);  
  
    for (int i = 0; i < xs.size(); i++) {  
        int x = xs.get(i);  
        sum += x;  
        System.out.println("i=" + i + " x=" + x + " sum=" + sum);  
    }  
  
    return sum / xs.size();  
}
```

Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    System.out.println("xs = " + xs);  
  
    for (int i = 0; i < xs.size(); i++) {  
        int x = xs.get(i);  
        sum += x;  
        System.out.println("i=" + i + " x=" + x + " sum=" + sum);  
    }  
  
    System.out.println("final sum=" + sum + " size=" + xs.size());  
  
    return sum / xs.size();  
}
```

Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    System.out.println("xs = " + xs);  
  
    for (int i = 0; i < xs.size(); i++) {  
        int x = xs.get(i);  
        sum += x;  
        System.out.println("i=" + i + " x=" + x + " sum=" + sum);  
    }  
  
    System.out.println("final sum=" + sum + " size=" + xs.size());  
    System.out.println("sum / size (int division) = " + (sum / xs.size()));  
  
    return sum / xs.size();  
}
```

Logging.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    System.out.println("xs = " + xs);  
  
    for (int i = 0; i < xs.size(); i++) {  
        int x = xs.get(i);  
        sum += x;  
        System.out.println("i=" + i + " x=" + x + " sum=" + sum);  
    }  
  
    System.out.println("final sum=" + sum + " size=" + xs.size());  
    System.out.println("sum / size (int division) = " + (sum / xs.size()));  
  
    return sum / (double) xs.size();  
}
```

Traditional debugger.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    for (int i = 0; i < xs.size(); i++)  
        sum += xs.get(i);  
    return sum / xs.size();  
}
```

Traditional debugger.

```
static double average(List<Integer> xs) {  
    int sum = 0;  
    for (int i = 0; i < xs.size(); i++)  
        sum += xs.get(i);  
    return sum / xs.size();  
}
```

The classic debugging cycle.

The classic debugging cycle.

1. Observe the bug, reproduce it

The classic debugging cycle.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"

The classic debugging cycle.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)

The classic debugging cycle.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)
4. Analyze the output to confirm or refute your hypothesis

The classic debugging cycle.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)
4. Analyze the output to confirm or refute your hypothesis
5. If confirmed, fix the bug. If refuted, form a new hypothesis and repeat ↺

The classic debugging cycle.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)
4. Analyze the output to confirm or refute your hypothesis
5. If confirmed, fix the bug. If refuted, form a new hypothesis and repeat ↺

? What happens if the developer can't find a viable hypothesis?

Example #2.

Example #2.

- I put a neutral cheese batch in a cellar

Example #2.

- I put a neutral cheese batch in a cellar
- Each day (turn), a random effect happens to the batch (good or bad)

Example #2.

- I put a neutral cheese batch in a cellar
- Each day (turn), a random effect happens to the batch (good or bad)
- At the end, we ask a cheese maker (the classifier) what cheese we produced

Example #2.

- I put a neutral cheese batch in a cellar
- Each day (turn), a random effect happens to the batch (good or bad)
- At the end, we ask a cheese maker (the classifier) what cheese we produced



Example #2 : Batch attributes.

Example #2 : Batch attributes.

- **humidity** and **temperature** of the cellar

Example #2 : Batch attributes.

- **humidity** and **temperature** of the cellar
- **rindIntegrity**

Example #2 : Batch attributes.

- **humidity** and **temperature** of the cellar
- **rindIntegrity**
- **moldRisk**

Example #2 : Batch attributes.

- **humidity** and **temperature** of the cellar
- **rindIntegrity**
- **moldRisk**
- **salt**

Example #2 : Batch attributes.

- **humidity** and **temperature** of the cellar
- **rindIntegrity**
- **moldRisk**
- **salt**
- temporary status:
 - **brineBarrier** (absorbs incoming mold growth)
 - **contaminationStacks** (amplifies mold growth)

Example #2 : Turn logic.

Each turn:

Example #2 : Turn logic.

Each turn:

1. passive decay + background rules (`tickStatus`)

Example #2 : Turn logic.

Each turn:

1. passive decay + background rules (`tickStatus`)
2. apply one **Effect** at random

Example #2 : Turn logic.

Each turn:

1. passive decay + background rules (`tickStatus`)
2. apply one **Effect** at random
3. clamp batch attributes to valid ranges

Example #2 : Turn logic.

Each turn:

1. passive decay + background rules (`tickStatus`)
2. apply one **Effect** at random
3. clamp batch attributes to valid ranges
4. record cellar history (moving averages)

Example #2 : Effects.

Each turn:

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE( $\pm$ points)` → Change the humidity

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE( $\pm$ points)` → Change the humidity
- `TEMP_CHANGE( $\pm$ °C)` → Change the temperature

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE( $\pm$ points)` → Change the humidity
- `TEMP_CHANGE( $\pm^{\circ}\text{C}$ )` → Change the temperature
- `BRINE_WASH(+barrier)` → adds barrier + increases salt

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE(±points)` → Change the humidity
- `TEMP_CHANGE(±°C)` → Change the temperature
- `BRINE_WASH(+barrier)` → adds barrier + increases salt
- `TURN_CHEESE(effort)` → reduces mold risk

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE(±points)` → Change the humidity
- `TEMP_CHANGE(±°C)` → Change the temperature
- `BRINE_WASH(+barrier)` → adds barrier + increases salt
- `TURN_CHEESE(effort)` → reduces mold risk
- `CONTAMINATION(+stacks)` → increases future mold scaling

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE(±points)` → Change the humidity
- `TEMP_CHANGE(±°C)` → Change the temperature
- `BRINE_WASH(+barrier)` → adds barrier + increases salt
- `TURN_CHEESE(effort)` → reduces mold risk
- `CONTAMINATION(+stacks)` → increases future mold scaling
- `MOLD_GROWTH(raw, source)` → main damage pipeline

Example #2 : Effects.

Each turn:

- `HUMIDITY_CHANGE(±points)` → Change the humidity
- `TEMP_CHANGE(±°C)` → Change the temperature
- `BRINE_WASH(+barrier)` → adds barrier + increases salt
- `TURN_CHEESE(effort)` → reduces mold risk
- `CONTAMINATION(+stacks)` → increases future mold scaling
- `MOLD_GROWTH(raw, source)` → main damage pipeline
- `OPEN_DOOR(humiditySpikePer mille, tempDropC)` → sudden shock event

Example #2 : The classifier.

Example #2 : The classifier.

- The classifier uses:
 - `avgHumidity()` and `avgTemp()` over a sliding window
 - plus end-state: `rindIntegrity` , `moldRisk` , `salt`

Example #2 : The classifier.

- The classifier uses:
 - `avgHumidity()` and `avgTemp()` over a sliding window
 - plus end-state: `rindIntegrity` , `moldRisk` , `salt`
- It matches ranges:
 - **GRUYERE** wants: stable 10–14°C, 85–92% avg humidity, low mold risk, strong rind
 - **VACHERIN** wants: very humid, cool, softer rind
 - **RACLETTE** wants: humid bands + higher salt, controlled mold

Example #2 : The bug.

Example #2 : The bug.



After running many simulations, we notice that the classifier often misclassifies certain batches.

Example #2 : The bug.



After running many simulations, we notice that the classifier often misclassifies certain batches.



Is it possible to debug this simulation to identify and fix the misclassification issue with the two presented strategies?

Example #2 : Debugging.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)
4. Analyze the output to confirm or refute your hypothesis
5. If confirmed, fix the bug. If refuted, form a new hypothesis and repeat ↺

Example #2 : Debugging.

1. Observe the bug, reproduce it
2. Form a hypothesis about its cause: "I think the problem is here"
3. Add print statements or use a debugger to inspect the program state (add breakpoints)
4. Analyze the output to confirm or refute your hypothesis
5. If confirmed, fix the bug. If refuted, form a new hypothesis and repeat ↺

? Where the batch process breaks down?

Where the process breaks down?

```
package ch.epfl.printwizard.examples.cheese;

import java.util.*;

public final class Main {

    public static void main(String[] args) {
        long seed = 1234567890L;
        Simulation sim = new Simulation(seed);

        Batch batch = new Batch("Batch-01");
        List<Effect> effects = sim.generateEffects(50);

        for (int turn = 0; turn < effects.size(); turn++) {
            sim.applyEffect(batch, effects.get(turn), turn);
        }

        CheeseType cheese = CheeseClassifier.classify(batch);
        System.out.println("Final: " + batch.debugString());
        System.out.println("Resulting cheese: " + cheese);
        System.out.println("Why: " + CheeseClassifier.explain(batch, cheese));
    }
}
```

Trace-Based Debugging

Trace-Based Debugging

- *Record*: Instruments the program to record its execution as a trace.

Trace-Based Debugging

- *Record*: Instruments the program to record its execution as a trace.
- *Explore*: Inspects the program after execution to browse its trace.

Trace-Based Debugging

- *Record*: Instruments the program to record its execution as a trace.
- *Explore*: Inspects the program after execution to browse its trace.
- *Explain*: Connects observed bugs to the expressions and statements that caused them.

Trace-Based Debugging

- *Record*: Instruments the program to record its execution as a trace.
- *Explore*: Inspects the program after execution to browse its trace.
- *Explain*: Connects observed bugs to the expressions and statements that caused them.

The tracing debugger is called PrintWizard and works for Java programs.

Demo?

PrintWizard

PrintWizard

- Backend (Responsible for retrieving program execution trace)

PrintWizard

- Backend (Responsible for retrieving program execution trace)
- The recorded trace is saved in a set of structured files.

PrintWizard

- Backend (Responsible for retrieving program execution trace)
- The recorded trace is saved in a set of structured files.
- Frontend (User interface to explore and analyze the trace)

PrintWizard : First Version (2024)

PrintWizard : First Version (2024)

- Developed by Erwan Serandour in 2024 (Master's thesis)

PrintWizard : First Version (2024)

- Developed by Erwan Serandour in 2024 (Master's thesis)
- Laid the foundations of the tool, particularly at the backend level.

PrintWizard : First Version (2024)

- Developed by Erwan Serandour in 2024 (Master's thesis)
- Laid the foundations of the tool, particularly at the backend level.
- Basic frontend functionalities for trace exploration.

PrintWizard : First Version (2024)

- Developed by Erwan Serandour in 2024 (Master's thesis)
- Laid the foundations of the tool, particularly at the backend level.
- Basic frontend functionalities for trace exploration.

```

main
5  Car car = new Car("Toyota", 2005); car= Car$
6    new Person("Alice":Alice, 30:30, car: Car$) ← Person$
   <init>
   ...
6  Person alice = new Person("Alice", 30, car); alice= Person$
9  alice.setAge(31:31);
   setAge
   ...
10 alice.setName("Alice Smith":Alice Smith);
   setName
   ...
11 alice.getCar().setModel("Honda":Honda);
11 alice.getCar() ← Car$ .setModel("Honda");
   getCar
   ...
   setModel
   ...
12 alice.getCar().setYear(2010:2010);
12 alice.getCar() ← Car$ .setYear(2010);
   getCar
   ...
   setYear
   ...
15 car.setModel("Ford":Ford);
   setModel
   ...
17 Car testCar = car; testCar= Car$
18 alice.setCar(new Car("Mazda", 2022): Car$);
   setCar
   ...

```

Person\$

- name = Alice
- age = 30
- car = Car\$

Car\$

- model = Toyota
- year = 2005

PrintWizard : Second Version (2025)

PrintWizard : Second Version (2025)

- Developed by Bastien Jolidon & Kelvin Kappeler in 2025 (Master's semester project)

PrintWizard : Second Version (2025)

- Developed by Bastien Jolidon & Kelvin Kappeler in 2025 (Master's semester project)
- Improves the frontend without touching the backend.

PrintWizard : Second Version (2025)

- Developed by Bastien Jolidon & Kelvin Kappeler in 2025 (Master's semester project)
- Improves the frontend without touching the backend.

```
Garage.java / main()

main()
5 > Car car = new Car("Toyota", 2005);
6 > Person alice = new Person("Alice", 30, car);
6 > <init>(string: Alice, int: 30, Car: $060) -> Person: $423
6 > Person alice := Person: $423
9 > alice.setAge(31);
10 > alice.setName("Alice Smith");
11 > alice.getCar().setModel("Honda");
12 > alice.getCar().setYear(2010);
15 > car.setModel("Ford");
17 > Car testCar = car;
18 > alice.setCar(new Car("Mazda", 2022));
```

Object Inspector  x

Car: \$060 (v5)

string model: Toyota
Toyota | v1 | Car: \$060 

▶ Toyota | v2 | Car: \$060 

Honda | v21 | Car: \$060 

Ford | v25 | Car: \$060 

int year: 2005
2005 | v1 | Car: \$060 

▶ 2005 | v2 | Car: \$060 

2010 | v25 | Car: \$060 

Object Inspector  x

Car: \$763 (v27)

string model: Mazda

int year: 2022

PrintWizard : Problems encountered

PrintWizard : Problems encountered

- No history / instrumentation of mutable objects.

PrintWizard : Problems encountered

- No history / instrumentation of mutable objects.
- No support for some basic Java features (i++).

PrintWizard : Problems encountered

- No history / instrumentation of mutable objects.
- No support for some basic Java features (i++).
- Recorded trace is hard to explore and misses important information.

PrintWizard : Problems encountered

- No history / instrumentation of mutable objects.
- No support for some basic Java features (i++).
- Recorded trace is hard to explore and misses important information.
- Too much work in the frontend to reconstruct correctly the program execution.

PrintWizard : Problems encountered

- No history / instrumentation of mutable objects.
- No support for some basic Java features (i++).
- Recorded trace is hard to explore and misses important information.
- Too much work in the frontend to reconstruct correctly the program execution.
- Heavy setup.

Thesis goals

Thesis goals

- Create a new version of PrintWizard that addresses the problems encountered in the previous versions.

Thesis goals

- Create a new version of PrintWizard that addresses the problems encountered in the previous versions.
- Be able to use PrintWizard in an academic context.

Thesis questions

Thesis questions

Trace fidelity

? How can the trace be recorded to explain values and control flows while respecting Java semantics?

Thesis questions

Thesis questions

Trace structure

? What data format model would improve and facilitate the frontend's work?

Thesis questions

Thesis questions

Usability

? What user interface and frontend features would make the trace easily explorable and efficient?

Contents

Contents

- Main Architecture

Contents

- Main Architecture
- Trace Format

Contents

- Main Architecture
- Trace Format
- Backend Architecture
 - Static Extraction
 - Instrumentation (Java Agent + Javac Plugin)

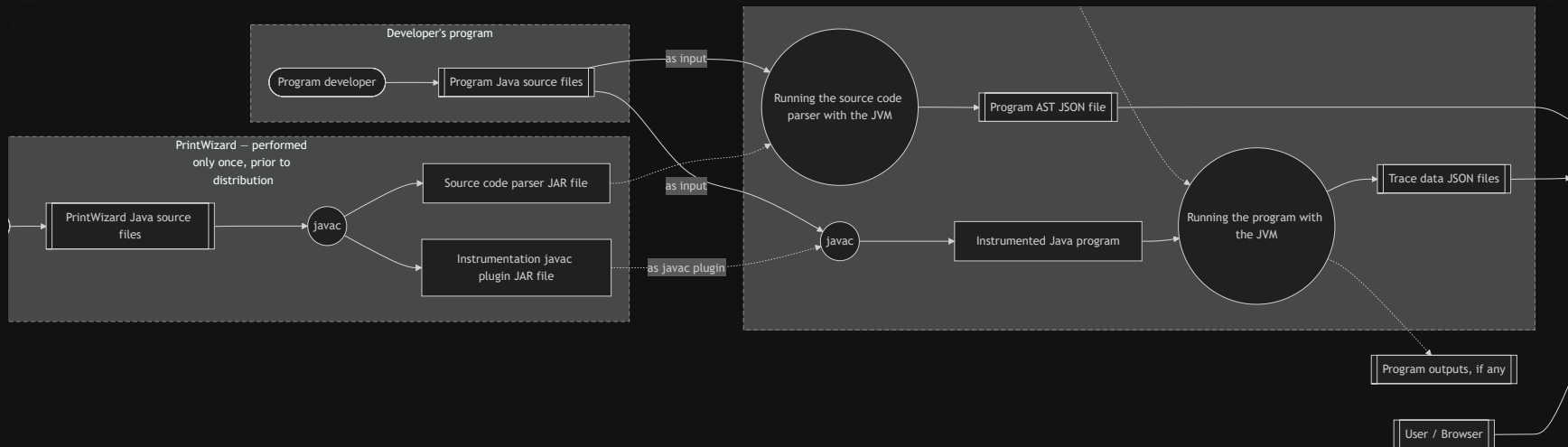
Contents

- Main Architecture
- Trace Format
- Backend Architecture
 - Static Extraction
 - Instrumentation (Java Agent + Javac Plugin)
- Frontend Architecture

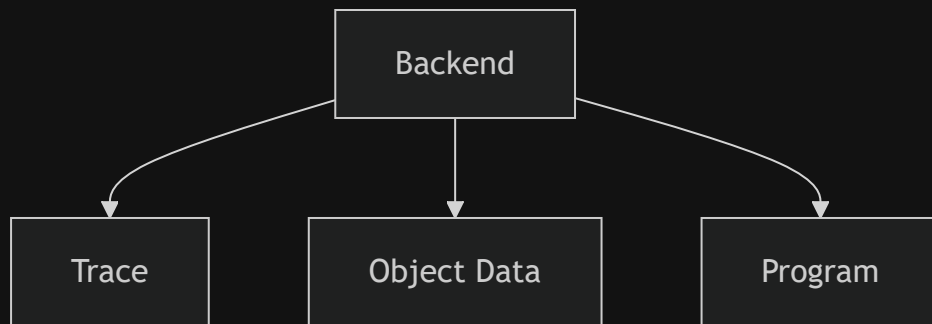
Contents

- Main Architecture
- Trace Format
- Backend Architecture
 - Static Extraction
 - Instrumentation (Java Agent + Javac Plugin)
- Frontend Architecture
- Debug Cheese Simulation

Main Architecture of PrintWizard



Previous Trace Format



Short program example

```
public class Main {  
  
    public static void main(String[] args) {  
        int i = 3 + 2;  
  
        if (i > 4) {  
            System.out.println("i is greater than 4");  
        }  
  
        Player player = new Player(100);  
        player.setHealthPoints(80);  
    }  
  
    public static class Player {  
        private int healthPoints;  
  
        public Player(int healthPoints) {  
            this.healthPoints = healthPoints;  
        }  
  
        public void setHealthPoints(int healthPoints) {  
            this.healthPoints = healthPoints;  
        }  
    }  
}
```

Trace

```
{
  "trace": [
    {
      "eventId": 0,
      "eventType": "controlFlow",
      "type": "GroupEvent",
      "pos": "start",
      "kind": {
        "type": "FunctionContext",
        "functionName": "main"
      }
    },
    {
      "eventId": 1,
      "eventType": "statement",
      "type": "GroupEvent",
      "pos": "start"
    },
    {
      "eventId": 2,
      "eventType": "subStatement",
      "type": "GroupEvent"
```

Object Data

```
[
  {
    "self": {
      "className": {
        "className": "Player",
        "packageName": ""
      },
      "pointer": 295530567,
      "version": 1,
      "dataType": "instanceRef"
    },
    "fields": [
      {
        "identifier": {
          "owner": {
            "className": {
              "className": "Player",
              "packageName": ""
            },
            "pointer": 295530567,
            "version": 1,
            "dataType": "instanceRef"
          }
        }
      }
    ]
  }
]
```

Program

```
{
  "sourceFile": {
    "fileName": "Main.java",
    "packageName": ""
  },
  "syntaxNodes": {
    "Main.java-3:29-3:42": {
      "endLine": 3,
      "identifier": "Main.java-3:29-3:42",
      "expression": {
        "kind": "expression",
        "tokens": [
          {
            "kind": "Text",
            "text": "String[] args"
          }
        ]
      }
    },
    "children": [],
    "kind": "presentInSourceCode",
    "prefix": {
      "kind": "Text"
```

Inspirations

Inspirations

- Google Trace Event Format (TEF)

```
{
  "displayTimeUnit": "ms",
  "traceEvents": [
    { "name": "process_name", "ph": "M", "pid": 100, "tid": 0, "args": { "name": "my_app" } },
    { "name": "thread_name", "ph": "M", "pid": 100, "tid": 1, "args": { "name": "main" } },

    { "name": "LoadConfig", "cat": "startup", "ph": "X",
      "ts": 1200000, "dur": 3500, "pid": 100, "tid": 1,
      "args": { "path": "/etc/my_app/config.json" }
    },

    { "name": "CacheMiss", "cat": "cache", "ph": "i",
      "ts": 1202000, "pid": 100, "tid": 1,
      "s": "t",
      "args": { "key": "user:42" }
    }
  ]
}
```

Inspirations

Inspirations

- Birdseye

```
i = 1
s = 0

if ( i == 1 ):
    s += 1
else:
    s -= 1
```

└─ ✓ i == 1 = bool: True

Inspirations

Inspirations

- Python Tutor

The screenshot displays the Python Tutor interface for Python 3.6. On the left, a code editor shows a script with seven lines. Line 2, `s = 0`, is highlighted with a green arrow, indicating it was just executed. Line 4, `if (i == 1):`, is highlighted with a red arrow, indicating it is the next line to execute. The code is as follows:

```
1 i = 1
2 s = 0
3
4 if (i == 1):
5     s += 1
6 else:
7     s -= 1
```

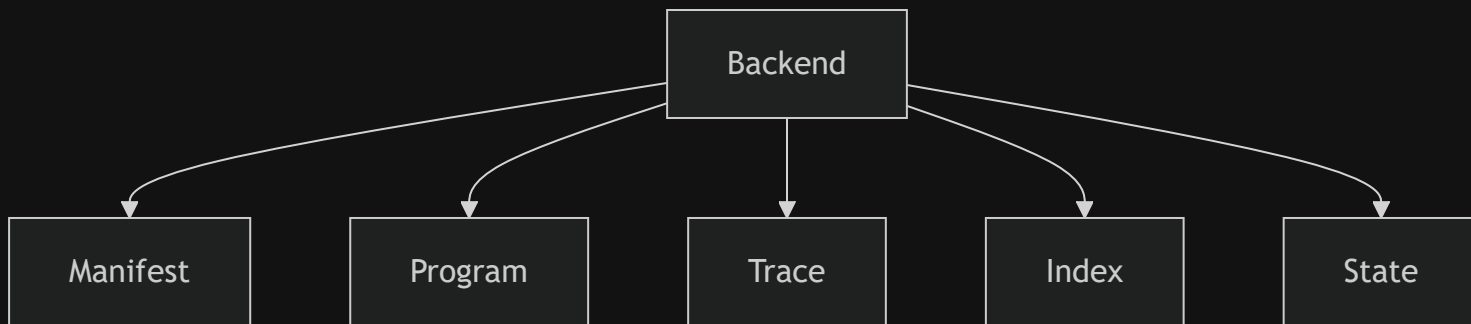
Below the code editor is a legend: a green arrow for 'line that just executed' and a red arrow for 'next line to execute'. A progress bar shows the current step, and navigation buttons include '<< First', '< Prev', 'Next >', and 'Last >>'. The status 'Step 3 of 4' is shown. A link 'Edit Code & Get AI Help' is also present.

On the right, the 'Frames' pane shows the 'Global frame' with two variables: `i` with value `1` and `s` with value `0`. The 'Objects' pane is currently empty.

Visualized with pythontutor.com

New Trace Format

New Trace Format



Manifest

```
{  
  "version" : "1.0.0",  
  "generatedAt" : "2026-01-28T18:02:00.057755700",  
  "fileLocations" : {  
    "programFilePath" : "program.json",  
    "traceFilePath" : "trace.json",  
    "indexFilePath" : "index.json",  
    "stateFilePath" : "state.json"  
  }  
}
```

Program

```
{
  "sources" : [ {
    "sourceId" : "src:ch/epfl/printwizard/examples/example/Main.java",
    "path" : "Examples/src/main/java/ch/epfl/printwizard/examples/example/Main.java",
    "language" : "java",
    "lines" : 15
  }, {
    "sourceId" : "src:ch/epfl/printwizard/examples/example/Player.java",
    "path" : "Examples/src/main/java/ch/epfl/printwizard/examples/example/Player.java",
    "language" : "java",
    "lines" : 13
  } ],
  "classes" : [ {
    "id" : "cls:ch.epfl.printwizard.examples.example.Main",
    "name" : "Main",
    "packageName" : "ch.epfl.printwizard.examples.example",
    "sourceId" : "src:ch/epfl/printwizard/examples/example/Main.java"
  }, {
    "id" : "cls:ch.epfl.printwizard.examples.example.Player",
    "name" : "Player",
    "packageName" : "ch.epfl.printwizard.examples.example",
    "sourceId" : "src:ch/epfl/printwizard/examples/example/Player.java"
```

Trace

```
{
  "spans" : [ {
    "spanId" : "spn:1",
    "parentSpanId" : "null",
    "methodId" : "m:ch.epfl.printwizard.examples.example.Main.main(String[])void",
    "thisRef" : null,
    "args" : [ {
      "name" : "args",
      "value" : {
        "value" : null,
        "valueObjectId" : "obj:1",
        "type" : "java.lang.String[]",
        "kind" : "ARRAY",
        "objectVersion" : null
      },
      "type" : "String[]"
    } ],
    "startEventId" : "eve:1",
    "endEventId" : "eve:13",
    "startLoc" : {
      "sourceId" : "src:ch/epfl/printwizard/examples/example/Main.java",
      "line" : 5
    }
  } ]
}
```

Index

```
{
  "byLine" : {
    "src:ch/epfl/printwizard/examples/example/Player.java" : {
      "11" : [ "eve:11" ],
      "6" : [ "eve:7" ],
      "7" : [ "eve:8" ],
      "8" : [ "eve:9" ],
      "10" : [ "eve:10" ]
    },
    "src:ch/epfl/printwizard/examples/example/Main.java" : {
      "12" : [ "eve:6" ],
      "5" : [ "eve:1" ],
      "6" : [ "eve:2" ],
      "8" : [ "eve:3", "eve:4" ],
      "9" : [ "eve:5" ]
    }
  },
  "byLocal" : {
    "local:player@src:ch/epfl/printwizard/examples/example/Main.java:250#spn:1" : [ "eve:6" ],
    "local:i@src:ch/epfl/printwizard/examples/example/Main.java:130#spn:1" : [ "eve:2" ]
  },
  "byObject" : {
```

State

```
{
  "objects" : {
    "obj:2" : {
      "objectId" : "obj:2",
      "type" : "ch.epfl.printwizard.examples.example.Player",
      "timeline" : [ {
        "version" : 1,
        "eventId" : "eve:8",
        "fields" : {
          "healthPoints" : {
            "type" : "int",
            "value" : 100,
            "objectId" : null
          }
        }
      }
    ], {
      "version" : 2,
      "eventId" : "eve:11",
      "fields" : {
        "healthPoints" : {
          "type" : "int",
          "value" : 80
        }
      }
    }
  }
}
```

Backend

Backend

Static Extraction

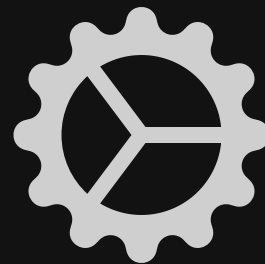


Backend

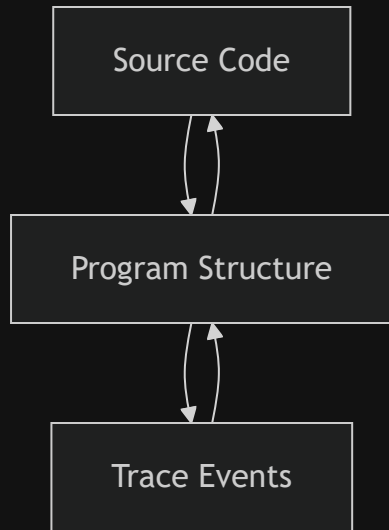
Static Extraction



Instrumentation



Static Extraction



Static Extraction

Static Extraction

- Java Parser

JP

Static Extraction

- Java Parser

JP



Java Parser does not know if the code has been executed or not.

Instrumentation

Instrumentation

- Modify the behaviour of the program to collect runtime data.

Instrumentation

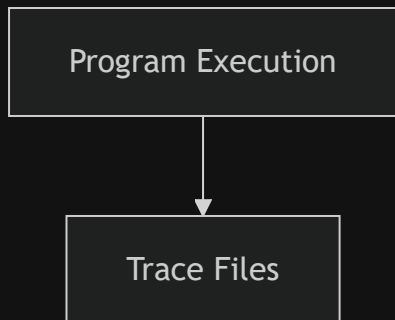
- Modify the behaviour of the program to collect runtime data.

? Why do we need instrumentation?

Instrumentation

- Modify the behaviour of the program to collect runtime data.

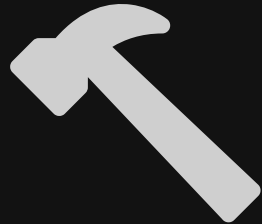
? Why do we need instrumentation?



Instrumentation

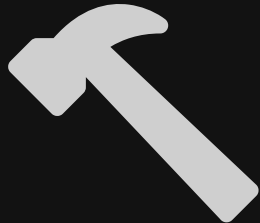
Instrumentation

Java Agent

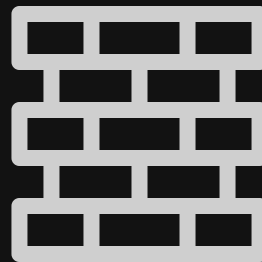


Instrumentation

Java Agent

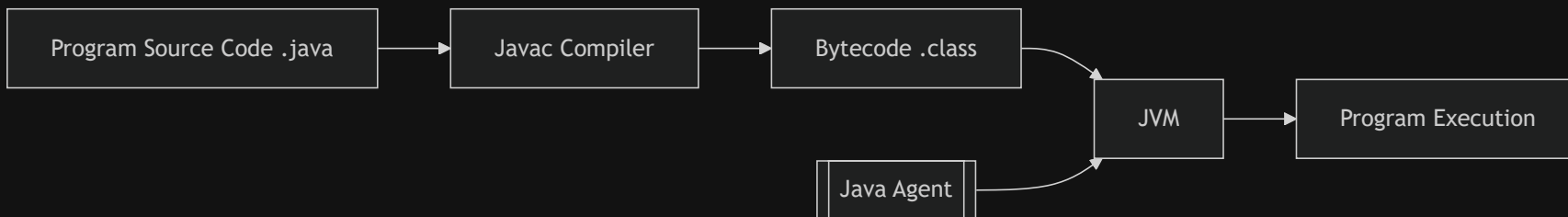


Javac Plugin

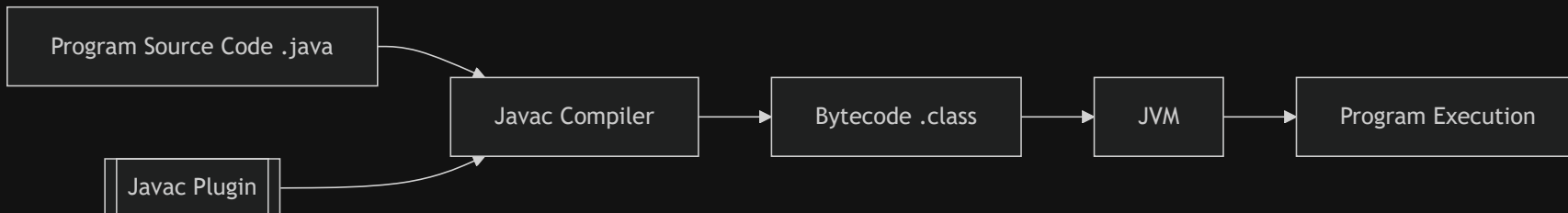
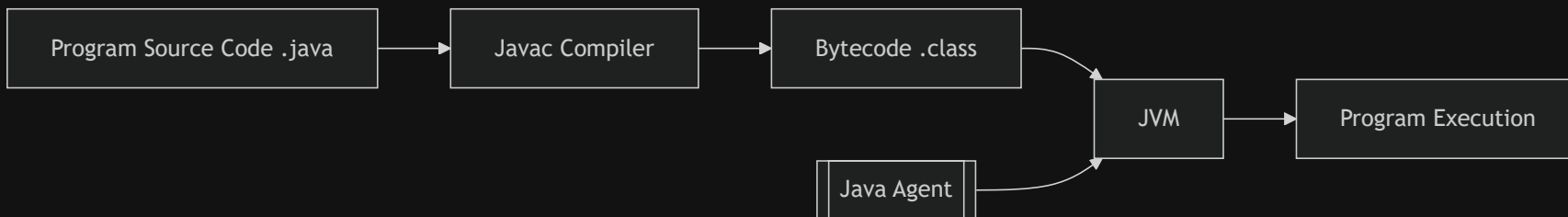


Instrumentation Toolchain

Instrumentation Toolchain



Instrumentation Toolchain



Java Agent

Java Agent

- Runtime instrumentation of Java bytecode

Java Agent

- Runtime instrumentation of Java bytecode
- Couple with ASM library for bytecode manipulation



Java Agent

- Runtime instrumentation of Java bytecode
- Couple with ASM library for bytecode manipulation



- Does not require the source code

Java Agent

Java Agent

? Why I started with Java Agent?

Java Agent

? Why I started with Java Agent?

- I did not check enough why the flaws of the first backend (Javac Plugin) were happening.

How to instrument with Java Agent?

How to instrument with Java Agent?

- Detect a specific bytecode instruction (e.g., the addition for $3 + 5$)

How to instrument with Java Agent?

- Detect a specific bytecode instruction (e.g., the addition for $3 + 5$)
- Inject a call to our own method when that instruction is executed

How to instrument with Java Agent?

- Detect a specific bytecode instruction (e.g., the addition for $3 + 5$)
- Inject a call to our own method when that instruction is executed
- Our method will log the necessary data to a trace file

How to instrument with Java Agent?

- Detect a specific bytecode instruction (e.g., the addition for $3 + 5$)
- Inject a call to our own method when that instruction is executed
- Our method will log the necessary data to a trace file

```
public class Main {  
  
    public static void main(String[] args) {  
        int i = 3 + 5;  
    }  
  
}
```

How to instrument with Java Agent?

- Detect a specific bytecode instruction (e.g., the addition for $3 + 5$)
- Inject a call to our own method when that instruction is executed
- Our method will log the necessary data to a trace file

```
public class Main {  
  
    public static void main(String[] args) {  
        int i = 3 + 5;  
    }  
  
}
```

```
ICONST_3  
ICONST_5  
IADD  
ISTORE
```

How to instrument with Java Agent?



How to instrument with Java Agent?

```
@Override
public void visitVarInsn(int opcode, int var) {
    switch (opcode) {
        case ISTORE, LSTORE, FSTORE, DSTORE, ASTORE : {
            String desc = switch (opcode) {
                case ISTORE -> "I";
                case LSTORE -> "J";
                case FSTORE -> "F";
                case DSTORE -> "D";
                default -> "Ljava/lang/Object;";
            };

            // long and double take two slots
            if (opcode == LSTORE || opcode == DSTORE) {
                dup2();
            } else {
                dup();
            }

            super.visitVarInsn(opcode, var);

            Type t = Type.getType(desc);
```

Why Javac Plugin was not working?

Why Javac Plugin was not working?

- Difficult, many edge cases and low-level APIs to manipulate ASTs and bytecode
 - Some types take two slots (long, double) in the stack.
 - Different bytecode instructions for different types (`ISTORE` , `ASTORE` , `FSTORE` , `DSTORE` , `LSTORE`)

Why Javac Plugin was not working?

- Difficult, many edge cases and low-level APIs to manipulate ASTs and bytecode
 - Some types take two slots (long, double) in the stack.
 - Different bytecode instructions for different types (ISTORE , ASTORE , FSTORE , DSTORE , LSTORE)
- The results of conditions often do not exist at runtime.

```
if (x > 10) { ... }  
if ("hello".length() > 5) { ... }
```

Why Javac Plugin was not working?

- Difficult, many edge cases and low-level APIs to manipulate ASTs and bytecode
 - Some types take two slots (long, double) in the stack.
 - Different bytecode instructions for different types (ISTORE , ASTORE , FSTORE , DSTORE , LSTORE)
- The results of conditions often do not exist at runtime.

```
if (x > 10) { ... }  
if ("hello".length() > 5) { ... }
```

- Nearly impossible to distinguish between different kinds of structures (loops vs. conditionals).

Javac Plugin

Javac Plugin

- Compile-time instrumentation of Java source code
- Manipulate the Abstract Syntax Tree (AST) of the program

Javac Plugin

- Compile-time instrumentation of Java source code
- Manipulate the Abstract Syntax Tree (AST) of the program

```
int i = 5;
```

Javac Plugin

- Compile-time instrumentation of Java source code
- Manipulate the Abstract Syntax Tree (AST) of the program

```
int i = TraceOut.recordLocal(5, ...);
```

Javac Plugin

- Compile-time instrumentation of Java source code
- Manipulate the Abstract Syntax Tree (AST) of the program

```
int i = TraceOut.endLocal(TraceOut.beginLocal(...), 5);
```

Javac Plugin

Javac Plugin

```
int i = 5 + 3;
```

Javac Plugin

```
int i = TraceOut.endArithmetic(TraceOut.beginArithmetic("+", ...), 5 + 3);
```

JavaC Plugin

```
String arId = TraceOut.beginArithmetic("+", ...);

// left operand
TraceOut.beginArithmeticLeft();
int _l = TraceOut.endArithmeticLeft(arId, 5);

// right operand
TraceOut.beginArithmeticRight();
int _r = TraceOut.endArithmeticRight(arId, 3);

// compute once, then close the arithmetic event
int _res = _l + _r;
int i = TraceOut.endArithmetic(arId, _l, _r, _res);
```

Javac Plugin

```
// int i = 5 + 3;
String localId = TraceOut.beginLocal("i", ...);

String arId = TraceOut.beginArithmetic("+", ...);

// left operand
TraceOut.beginArithmeticLeft();
int _l = TraceOut.endArithmeticLeft(arId, 5);

// right operand
TraceOut.beginArithmeticRight();
int _r = TraceOut.endArithmeticRight(arId, 3);

// close arithmetic (compute once)
int _res = _l + _r;
int rhs = TraceOut.endArithmetic(arId, _l, _r, _res);

// close local write + produce assigned value
int i = TraceOut.endLocal(localId, rhs);
```

Main Challenges with Javac Plugin

Main Challenges with Javac Plugin

- Handling all edge cases of Java syntax and semantics

```
if (condition1 && condition2) {... }
```

```
this.healthPoints = 100;
```

```
healthPoints = 100
```

Main Challenges with Javac Plugin

- Handling all edge cases of Java syntax and semantics

```
if (condition1 && condition2) {... }
```

```
this.healthPoints = 100;
```

```
healthPoints = 100
```

- Correctly linking the instrumentation data with the static extraction

Linking Static Extraction and Instrumentation

Linking Static Extraction and Instrumentation

? How to link static extraction with the instrumentation data?

Linking Static Extraction and Instrumentation

? How to link static extraction with the instrumentation data?

- Use unique IDs for methods, variables, and structures
- These IDs are generated the same way in both the static extractor and the instrumentation tool

Linking Static Extraction and Instrumentation

? How to link static extraction with the instrumentation data?

- Use unique IDs for methods, variables, and structures
- These IDs are generated the same way in both the static extractor and the instrumentation tool

```
m:ch.epfl.printwizard.examples.example.Main.main(String[])void
```

```
src:ch/epfl/printwizard/examples/example/Player.java
```

Instrumented language constructs

Instrumented language constructs

- Local variable declarations and assignments

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments
- Object instantiations

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments
- Object instantiations
- Conditional statements (if / else if / else)

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments
- Object instantiations
- Conditional statements (if / else if / else)
- Loop statements (for / while / do while)

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments
- Object instantiations
- Conditional statements (if / else if / else)
- Loop statements (for / while / do while)
- Method calls

Instrumented language constructs

- Local variable declarations and assignments
- Field assignments
- Array creations and assignments
- Object instantiations
- Conditional statements (if / else if / else)
- Loop statements (for / while / do while)
- Method calls
- Return statements

Instrumented language constructs

Instrumented language constructs

- Arithmetic operations (+, -, *, /, %)

Instrumented language constructs

- Arithmetic operations (+, -, *, /, %)
- Comparison operations (==, !=, >, <, >=, <=)

Instrumented language constructs

- Arithmetic operations (+, -, *, /, %)
- Comparison operations (==, !=, >, <, >=, <=)
- Logical operations (&&, ||, !)

Instrumented language constructs

- Arithmetic operations (+, -, *, /, %)
- Comparison operations (==, !=, >, <, >=, <=)
- Logical operations (&&, ||, !)
- Unary operations (++ , --, unary -)

Instrumented language constructs

- Arithmetic operations (+, -, *, /, %)
- Comparison operations (==, !=, >, <, >=, <=)
- Logical operations (&&, ||, !)
- Unary operations (++, --, unary -)
- Assignment operations (=, +=, -=, *=, /=, %=)

Instrumented language constructs

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases
- Enhanced for-loops (for-each loops)

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases
- Enhanced for-loops (for-each loops)
- Switch statements

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases
- Enhanced for-loops (for-each loops)
- Switch statements
- Try-catch-finally blocks

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases
- Enhanced for-loops (for-each loops)
- Switch statements
- Try-catch-finally blocks
- Lambda expressions

Instrumented language constructs



What remains uninstrumented / too buggy to be used reliably?

- Ternary operators (condition ? expr1 : expr2)
 - 90% done, but do not work well in nested cases
- Enhanced for-loops (for-each loops)
- Switch statements
- Try-catch-finally blocks
- Lambda expressions
- Synchronized blocks and methods

It is time to debug the cheese simulator!

Future work: Backend Improvements

Future work: Backend Improvements

- Improve the Javac Plugin to handle more edge cases and language constructs

Future work: Backend Improvements

- Improve the Javac Plugin to handle more edge cases and language constructs
- Implementation of unit tests

Future work: Frontend Improvements

Future work: Frontend Improvements

- Implementation of other views / inspectors

Future work: Frontend Improvements

- Implementation of other views / inspectors
- Improve the usability and design of the UI (e.g., dark mode, keyboard shortcuts)

Future work: Setup and Documentation

Future work: Setup and Documentation

- Improve the setup process for developers

Future work: Setup and Documentation

- Improve the setup process for developers
- Create comprehensive documentation and tutorials for users

Conclusion for a (soon-to-be-no longer?) student

Conclusion for a (soon-to-be-no longer?) student

- Self-reflection on the challenges faced during the development of PrintWizard

Conclusion for a (soon-to-be-no longer?) student

- Self-reflection on the challenges faced during the development of PrintWizard
- Happy with the final results, despite the initial struggles

Conclusion for a (soon-to-be-no longer?) student

- Self-reflection on the challenges faced during the development of PrintWizard
- Happy with the final results, despite the initial struggles
- Gained valuable experience in Java instrumentation and program analysis

Conclusion for a (soon-to-be-no longer?) student

- Self-reflection on the challenges faced during the development of PrintWizard
- Happy with the final results, despite the initial struggles
- Gained valuable experience in Java instrumentation and program analysis
- Lessons learned about Java instrumentation

Conclusion for a (soon-to-be-no longer?) student

- Self-reflection on the challenges faced during the development of PrintWizard
- Happy with the final results, despite the initial struggles
- Gained valuable experience in Java instrumentation and program analysis
- Lessons learned about Java instrumentation
- PrintWizard has a clear path towards becoming a robust and user-friendly tool for Java program analysis.

Conclusion for PrintWizard

Conclusion for PrintWizard

- PrintWizard has a clear path towards becoming a robust and user-friendly tool for Java program analysis.

Conclusion for PrintWizard

- PrintWizard has a clear path towards becoming a robust and user-friendly tool for Java program analysis.
- Happy to contribute again

Acknowledgements

Acknowledgements

- Shardul Chiplunkar, my supervisor, for his guidance and support throughout the project.

Acknowledgements

- Shardul Chiplunkar, my supervisor, for his guidance and support throughout the project.
- Pr. Clément Pit-Claudel, for his valuable feedback and insights during the project.

Acknowledgements

- Shardul Chiplunkar, my supervisor, for his guidance and support throughout the project.
- Pr. Clément Pit-Claudel, for his valuable feedback and insights during the project.
- SYSTEMF lab, for providing the resources and environment to work on this project.

Questions?